PROJECT SUBMISSION

LegalEase AI

Date	29/09/2026
Team ID	SWTID - 2026 - 4944
Project Name	LegalEase AI
Maximum Marks	3 Marks

Project Executable Files

LegalEase

AI-Powered Legal Document Generator

Project Executable Files – Submission Document

This document provides a complete sample source-code set organized according to the uploaded Project Executable Files checklist. The LegalEase source specifies Streamlit + FastAPI + Gemini, editable preview, and TXT/DOCX/PDF export. ■filecite■turn1file1■L34-L52■

1. Submission Checklist

Item	LegalEase submission
Complete source code	Included below
README / Setup Guide	Included
requirements.txt	Included
Database schema / seed	N/A – not specified in source
Environment configuration	.env.example included
Deployed URL	Not specified / not deployed
APK / executable binary	N/A – web application
Dockerfile	Not specified in source
Test files/results	Basic API test file included
Demo video	Not specified

The uploaded submission template asks for source code, README, dependency file, environment configuration, tests, deployment details and run instructions. ■filecite■turn1file0■L12-L25■

2. Project Analysis Used for the Executable Set

The LegalEase source describes a Streamlit frontend, FastAPI backend, Gemini integration in ai_core/gemini_generator.py, and formatting modules for DOCX, PDF and TXT. ■filecite■turn1file1■L38-L52■

The documented user inputs are document type, parties, terms and effective date; the UI includes Generate, Edit, and download options. ■filecite■turn1file1■L123-L141■

The project documentation also specifies local testing of generation, preview, editing, and TXT/DOCX/PDF downloads. ■filecite■turn1file1■L278-L293■

The code below is a consolidated executable sample based on those documented requirements. Items not specified by the source, such as a real deployed URL, credentials, database, Dockerfile, and demo video, are explicitly marked rather than invented.

FILE: project_structure.txt

```
LegalEase/
|-- app.py
|-- main.py
|-- routes.py
|-- requirements.txt
|-- .env.example
|-- README.md
|-- ai_core/
|   |-- __init__.py
|   `-- gemini_generator.py
|-- utils/
|   |-- __init__.py
|   `-- formatters.py
`-- tests/
    `-- test_api.py
```

FILE: requirements.txt

```
fastapi
uvicorn
streamlit
python-docx
fpdf
Pillow
requests
google-generativeai
python-dotenv
pytest
```

FILE: .env.example

```
GEMINI_API_KEY=your_gemini_api_key_here
BACKEND_URL=http://127.0.0.1:8000
```

FILE: main.py

```
from fastapi import FastAPI
from routes import router

app = FastAPI(
    title="LegalEase API",
    description="AI-powered legal document generation API",
    version="1.0.0",
)

app.include_router(router)

@app.get("/")
def root():
    return {
        "status": "ok",
        "project": "LegalEase",
        "message": "LegalEase FastAPI backend is running"
    }
```

FILE: routes.py

```
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel, Field
from ai_core.gemini_generator import GeminiDocumentGenerator

router = APIRouter()
generator = GeminiDocumentGenerator()

class DocumentRequest(BaseModel):
    document_type: str = Field(..., min_length=2)
    parties: str = Field(..., min_length=2)
    terms: str = Field(..., min_length=2)
    dates: str = Field(..., min_length=2)

@router.post("/generate")
def generate_document(request: DocumentRequest):
    try:
        text = generator.generate_document(
            document_type=request.document_type,
            parties=request.parties,
            terms=request.terms,
            dates=request.dates,
        )
        return {
            "success": True,
            "document_type": request.document_type,
            "content": text
        }
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))
```

FILE: ai_core/__init__.py

```
# LegalEase AI core package
```

FILE: ai_core/gemini_generator.py

```
import os
import google.generativeai as genai
from dotenv import load_dotenv

load_dotenv()

class GeminiDocumentGenerator:
    def __init__(self):
        api_key = os.getenv("GEMINI_API_KEY")
        if not api_key:
            raise RuntimeError(
                "GEMINI_API_KEY is missing. Add it to your .env file."
            )
        genai.configure(api_key=api_key)
        self.model = genai.GenerativeModel("gemini-1.5-pro")

    def generate_document(self, document_type, parties, terms, dates):
        prompt = f'''
You are an AI legal-document drafting assistant.
Create a structured draft for the requested document.

Document type: {document_type}
Parties involved: {parties}
Terms and conditions: {terms}
Effective date: {dates}

Requirements:
- Use clear formal sections.
- Include title, parties, effective date, definitions where useful,
  terms/clauses, responsibilities, termination, and signature areas
  when appropriate to the document type.
- Do not invent personal facts that were not supplied.
- Return plain text suitable for preview and later DOCX/PDF formatting.
- State that the result is an AI-generated draft requiring legal review.
'''
        response = self.model.generate_content(prompt)
        return response.text.strip()
```

FILE: utils/__init__.py

```
# LegalEase utility package
```

FILE: utils/formatters.py

```
import re
from io import BytesIO
from docx import Document
from docx.shared import Pt
from fpdf import FPDF

def sanitize_text(text: str) -> str:
    replacements = {
        "\u2018": "'", "\u2019": "'", "\u201c": '"',
        "\u201d": '"', "\u2013": "-", "\u2014": "-"
    }
    for old, new in replacements.items():
        text = text.replace(old, new)
    return re.sub(r"[\x00-\x08\x0b\x0c\x0e-\x1f]", "", text)

def format_docx(text: str, doc_type: str) -> bytes:
    doc = Document()
    style = doc.styles["Normal"]
    style.font.name = "Times New Roman"
    style.font.size = Pt(11)

    title = doc.add_paragraph()
    run = title.add_run(doc_type.upper())
    run.bold = True
    run.font.size = Pt(16)

    for block in sanitize_text(text).split("\n"):
        block = block.strip()
        if block:
            p = doc.add_paragraph(block)
            p.paragraph_format.space_after = Pt(6)

    doc.add_paragraph(
        "AI-generated draft. Review by a qualified legal professional is recommended."
    )
    buffer = BytesIO()
    doc.save(buffer)
    return buffer.getvalue()

def format_pdf(text: str, doc_type: str) -> bytes:
    pdf = FPDF()
    pdf.set_auto_page_break(auto=True, margin=15)
    pdf.add_page()
    pdf.set_font("Arial", "B", 16)
    pdf.cell(0, 10, doc_type.upper(), ln=True, align="C")
    pdf.ln(4)
    pdf.set_font("Arial", size=11)
    for block in sanitize_text(text).split("\n"):
        block = block.strip()
        if block:
            pdf.multi_cell(0, 7, block)
            pdf.ln(2)
    pdf.ln(4)
    pdf.set_font("Arial", "I", 8)
    pdf.multi_cell(0, 5,
        "AI-generated draft. Review by a qualified legal professional is recommended."
    )
    return bytes(pdf.output(dest="S"))

def format_html_preview(text: str) -> str:
    safe = sanitize_text(text)
    return safe.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;").replace("\n", "<br>")
```

FILE: app.py

```
import os
import requests
import streamlit as st
from dotenv import load_dotenv
from utils.formatters import format_docx, format_pdf, format_html_preview

load_dotenv()

BACKEND_URL = os.getenv("BACKEND_URL", "http://127.0.0.1:8000")

st.set_page_config(page_title="LegalEase", page_icon="■■", layout="wide")

st.title("■■ LegalEase")
st.caption("AI-Powered Legal Document Generator")
st.info("LegalEase generates drafts for review; it is not a substitute for legal advice.")

document_type = st.text_input(
    "Document Type",
    placeholder="Freelance Work Contract / NDA / Lease Agreement"
)
parties = st.text_area(
    "Parties Involved",
    placeholder="Jane Doe (Service Provider), TechNova Inc. (Client)"
)
terms = st.text_area(
    "Terms & Conditions",
    placeholder="Payment within 30 days; Confidentiality; Termination with 15 days notice"
)
dates = st.text_input("Effective Date", placeholder="April 10, 2025")

if st.button("Generate Document", type="primary"):
    payload = {
        "document_type": document_type,
        "parties": parties,
        "terms": terms,
        "dates": dates,
    }
    try:
        response = requests.post(
            f"{BACKEND_URL}/generate",
            json=payload,
            timeout=120
        )
        if response.ok:
            data = response.json()
            st.session_state["content"] = data["content"]
            st.session_state["doc_type"] = document_type
        else:
            st.error(response.text)
    except requests.RequestException as exc:
        st.error(f"Backend connection failed: {exc}")

if "content" in st.session_state:
    st.subheader("Generated Document Preview")
    st.markdown(
        f'<div style="padding:20px;border-radius:10px;'
        f'background:#111827;color:white;">'
        f'{format_html_preview(st.session_state["content"])}</div>',
        unsafe_allow_html=True
    )

    edited = st.text_area(
        "Edit Document",
        value=st.session_state["content"],
        height=400
    )
    st.session_state["content"] = edited

    col1, col2, col3 = st.columns(3)
    with col1:
        st.download_button(
            "Download TXT",
            data=edited,
            file_name="legalease_document.txt",
            mime="text/plain"
        )
    with col2:
        st.download_button(
            "Download DOCX",
            data=format_docx(edited, st.session_state["doc_type"]),
            file_name="legalease_document.docx",
            mime="application/vnd.openxmlformats-officedocument.wordprocessingml.document"
        )
    with col3:
        st.download_button(
            "Download PDF",
            data=format_pdf(edited, st.session_state["doc_type"]),
            file_name="legalease_document.pdf",
```

```
    mime="application/pdf"
)
```

FILE: tests/test_api.py

```
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_root():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json()["project"] == "LegalEase"

def test_generate_validation():
    response = client.post("/generate", json={
        "document_type": "",
        "parties": "",
        "terms": "",
        "dates": ""
    })
    assert response.status_code == 422
```

FILE: README.md

```
# LegalEase – AI-Powered Legal Document Generator

## Project
LegalEase uses a Streamlit frontend, FastAPI backend, Gemini AI integration,
and document-formatting utilities to generate editable legal-document drafts.

## Folder Structure
LegalEase/
  app.py
  main.py
  routes.py
  requirements.txt
  .env.example
  README.md
  ai_core/gemini_generator.py
  utils/formatters.py
  tests/test_api.py

## Installation
1. Install Python 3.10+.
2. Create a virtual environment:
   python -m venv venv
3. Activate it on Windows:
   venv\Scripts\activate
4. Install dependencies:
   pip install -r requirements.txt
5. Copy .env.example to .env.
6. Add your Gemini API key:
   GEMINI_API_KEY=your_key_here

## Run Backend
uvicorn main:app --reload --port 8000

## Run Frontend
Open a second terminal and activate the same virtual environment:
streamlit run app.py

## Open
Frontend: http://localhost:8501
Backend: http://127.0.0.1:8000

## Test
pytest

## Usage
Enter document type, parties, terms and effective date.
Click Generate Document.
Review/edit the generated text.
Download TXT, DOCX or PDF.

## Deployment
The supplied project documentation lists options including Render, Railway,
Deta, Streamlit Community Cloud for frontend, and Fly.io/VPS for backend.
Configure environment variables on the selected platform.

## Security
Never commit the real .env file or API key to source control.

## Limitation
Generated documents are drafts and should be reviewed by a qualified legal
professional before real-world use.
```

3. Run Instructions

1. Create the folder structure shown in project_structure.txt.
2. Save each code block using the exact filename shown.
3. Install Python 3.10+ and create a virtual environment.
4. Run **pip install -r requirements.txt**.
5. Copy .env.example to .env and add the Gemini API key.
6. Start backend: **uvicorn main:app --reload --port 8000**.
7. Start frontend in another terminal: **streamlit run app.py**.
8. Open **http://localhost:8501** and generate a document.
9. Test with **pytest**.

The source documentation specifies Python 3.10+, FastAPI/Uvicorn, Streamlit, Google Generative AI SDK, python-docx, FPDF, Pillow, requests and python-dotenv. ■filecite■turn1file1■L142-L154■

4. Known Issues / Limitations

• A real Gemini API key is required for AI generation.
• No deployed URL or repository link was supplied in the source files.
• No database is specified in the supplied LegalEase documentation.
• The source describes Gemini 1.5 Pro; the executable sample therefore follows that documented model name.
• Generated legal documents are drafts and should be reviewed by a qualified legal professional before real-world use.